Siamese Networks

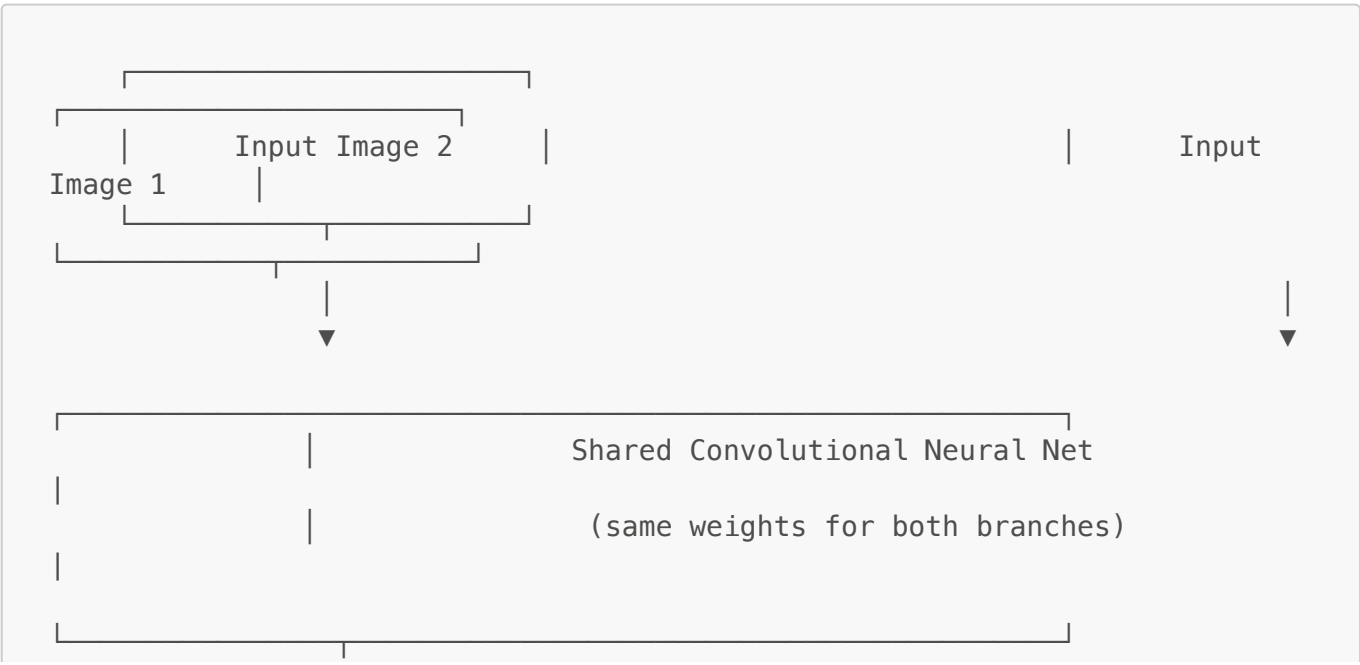
Skin Lesion Analysis: Melanoma Detection using The ISIC 2020 Challenge Dataset

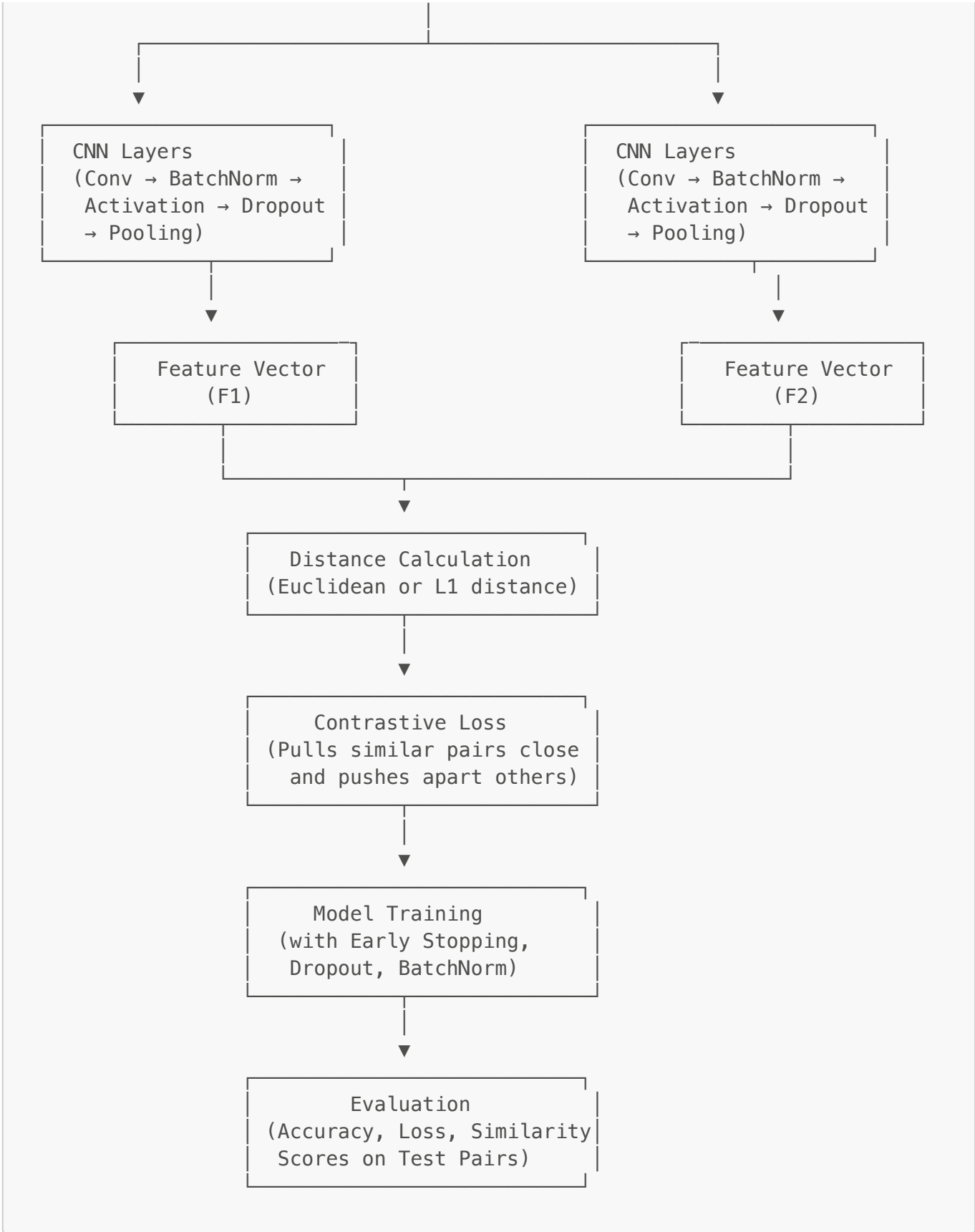
Author: Harshit Vishwa s47318063

Table of Contents

Introduction To Siamese Neural Network

A Siamese neural network is a neural network architecture that learns to measure the similarity between two inputs instead of classifying them. It uses two identical subnetworks with shared weights to compare pairs of inputs and output how alike they are. In my project, they are used to detect cancerous and non-cancerous skin lesions. The outputs are compared using a contrastive loss function, which pulls similar pairs closer together and pushes dissimilar pairs farther apart. This allows the network to learn a more meaningful feature representation of the input data.





Data Preperation

The DataLoader class constructs pairs of images for training the Siamese network.

Each pair consists of:

A positive pair: two images from the same class (benign–benign or malignant–malignant).

A negative pair: two images from different classes (benign–malignant).

Since the ISIC 2020 dataset is imbalanced (far more benign than malignant images), the pairing strategy had to be adjusted to prevent biased learning.

```
with open(self.output_path, "wb") as f:
    pickle.dump([[x1, x2], y], f)
```

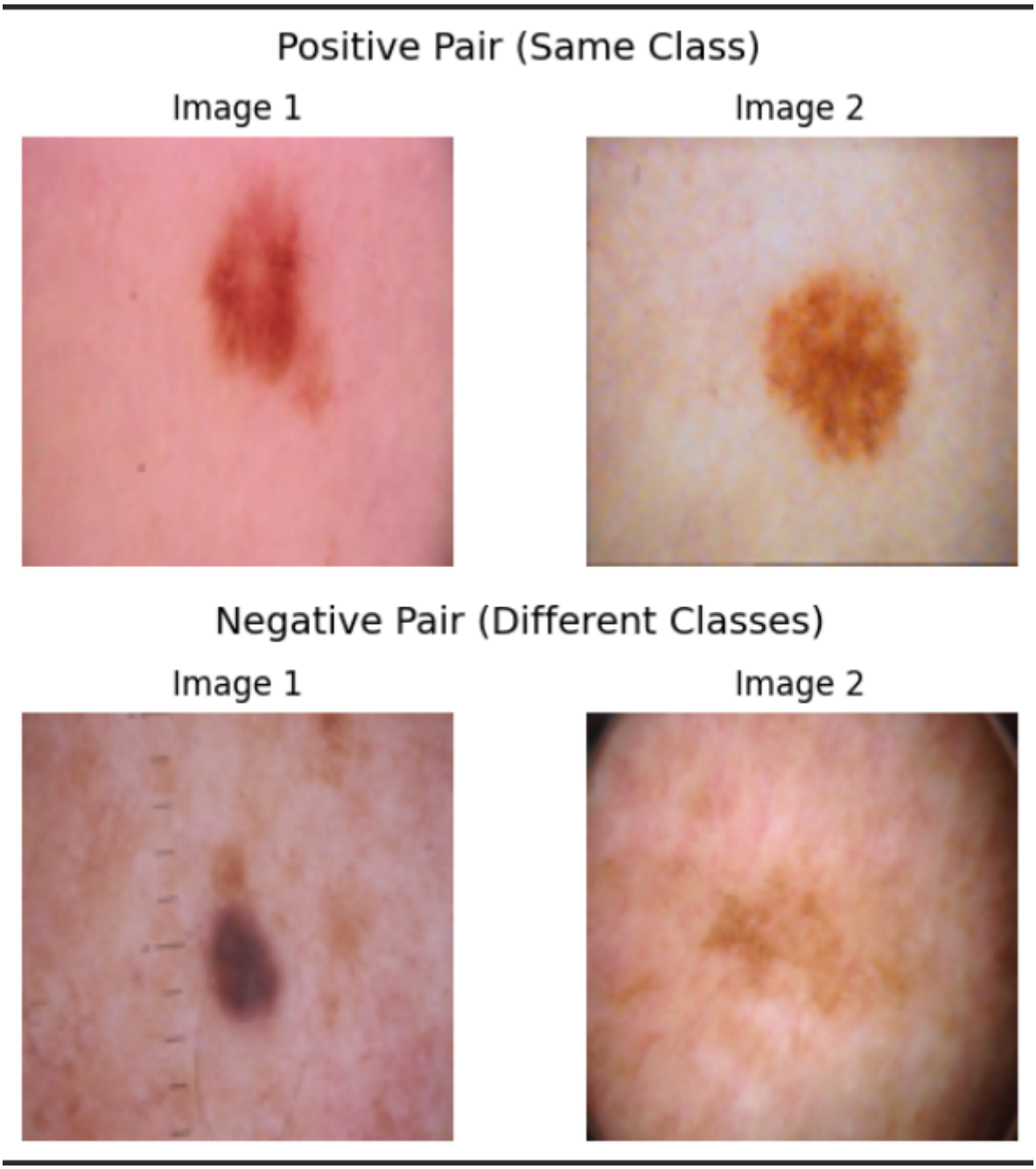
The _make_pairs() method:

1. Groups images by class (target = 0 benign, 1 malignant).
2. Randomly selects pairs to create both positive and negative examples.
3. Applies undersampling to balance the dataset — ensuring equal counts of positive and negative pairs.
4. Normalizes and resizes each image before saving to a .pkl file for faster loading in Colab.

Each image creates a positive pair with a random image of the same class and a negative pair with the opposite class. Then the positive and negative pairs are concantenanted together into pairs of variable corresponding to 0 or 1 labels.

```
x_first_balanced, x_second_balanced, y_balanced = self._make_pairs()
pickle.dump([[x_first_balanced, x_second_balanced], y_balanced], f)
```

The method takes in two inputs, x and y, and groups the images by class (malignant vs. benign). For each iteration, it selects a positive pair and then creates a negative pair. Both images are loaded, resized, and normalized, then stored in x_first, x_second, and y (representing the similarity score, 1 or 0). Finally, the data is converted into NumPy arrays.



Data Set

Dataset: ISIC 2020: Skin Lesion Analysis for Melanoma Detection

Input size: 105 × 105 × 3 (RGB)

Classes: Benign (0), Malignant (1)

Training/Validation Split: 80/20

Storage format: Pickled pairs (.pkl) for efficient reloading in Colab

Train and Validation split: To create the training and validation sets, the pairs of images were split 80/20, with 80% used for training and 20% reserved for model validation. This ensures sufficient data is available to accurately train the model and set aside a validation dataset to assess performance during training and identify potential overfitting.

Project Goals

1. Build a Siamese CNN for melanoma similarity learning with ≥ 85% accuracy
2. Mitigate dataset imbalance through pair balancing
3. Evaluate on unseen image pairs
4. Visualize feature embeddings and training metrics

File Structure

```
.venv/
Siamese_PatternRE_2025/
│
├── collab runs/
├── Siamese_PatternRE_2025/
├── Siamese_PatternRE_2025.git/
├── Siamese_PatternRE_2025.git.bfg-report
├── Siamese_PatternRE_2025.git.old/
│
├── README.md
├── Inital_training.txt
├── traininglog.txt
├── similarity_results_reference.csv
├── dataLoader.ipynb
│
├── dataset_siam.py
├── generate_pairs.py
├── generatepkltest.py
├── model_siam.py
├── plot_classification.py
├── plot_performance.py
├── predict_siam.py
├── train_siam.py
│
├── accuracy_curve.png
├── benchmarking.png
├── Distribution of similarity score.png
├── loss_curve.png
├── positivenegativepair.png
├── Top similarity scores.png
└── Validationaccuracy.png
```

Model Architecture

Based on: G. Koch, R. Zemel & R. Salakhutdinov (2015), “Siamese Neural Networks for One-Shot Image Recognition”. Paper link: https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf

- 4 × Convolutional Blocks (Conv → BN → ReLU → MaxPool)
- Dense(4096) feature embedding
- L1 distance layer for similarity
- Sigmoid output for binary similarity score
- Regularization techniques:
    - L2 regularization
    - Dropout layers
    - Batch Normalization
- Optimizer: Adam (learning rate = 5e-5)

Advantages Disadvandates Of Model

Advantages

1. Learns generalizable embeddings for unseen data
2. Works well with limited labeled samples
3. Captures subtle inter-class differences

Disadvantages

1. Requires careful pair generation
2. Slower convergence due to two-input architecture
3. Sensitive to imbalance in training pairs

Data Augmentation

All images were resized to **105×105** and normalized to the **0–1** range.
Augmentations included **rotation, flipping, and zooming** to improve generalization.

Although grayscale conversion was tested, the final model uses **RGB images**, as skin lesion classification relies heavily on color cues — subtle pigmentation, redness, or hue variations.
RGB preserves these diagnostic features, whereas grayscale removes valuable color information.

Training

The model was trained using **four convolutional blocks** followed by a **dense embedding layer**.
Dropout and **Batch Normalization** were introduced to reduce overfitting observed in initial experiments.
An **L1 distance layer** was used to compute similarity between the two image embeddings.

Training Configuration

- **Loss:** Binary Crossentropy
- **Optimizer:** Adam (learning rate = 0.00005)
- **Batch Size:** 32
- **Epochs:** 10
- **Early Stopping:** Enabled (patience = 5, min_delta = 0.01)

- **Pretrained Weights:** Supported via saved `.h5` files Training command:

```
!python siamese_train.py
```

Training Result

Initially, the model showed irregular and unstable loss reduction, indicating uneven exposure to positive and negative pairs. After implementing sampling with repetition, batches became more balanced—ensuring that each mini-batch contained a representative mix of both classes.

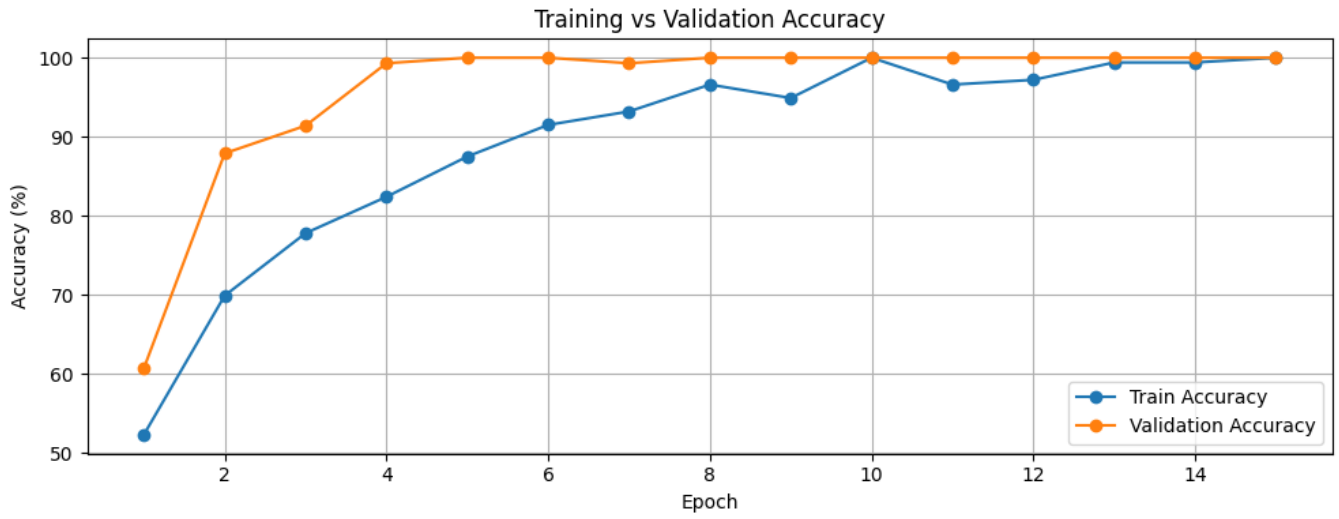
This adjustment stabilized the loss descent and prevented early overfitting. The final curve shows a steady downward trend with mild oscillations, reflecting a healthier convergence pattern. The remaining fluctuations are expected and correspond to stochastic batch variations caused by diverse image augmentations. Loss_curve.png

To address this:

- Sampling with repetition was introduced, ensuring more balanced exposure to positive and negative pairs across batches.
- Key hyperparameters such as learning rate, batch size, and margin threshold were fine-tuned.
- Augmentation diversity was increased, improving the representation of positive and negative examples.

Training accuracy improved progressively over epochs, plateauing below 80%. The visible fluctuations reflect the model's adaptation to the more diverse and challenging augmented samples introduced during retraining.

By tuning the learning rate and batch size, the model avoided overshooting and converged more smoothly compared to the initial runs, where accuracy jumped erratically or reached unrealistically high values due to data leakage.



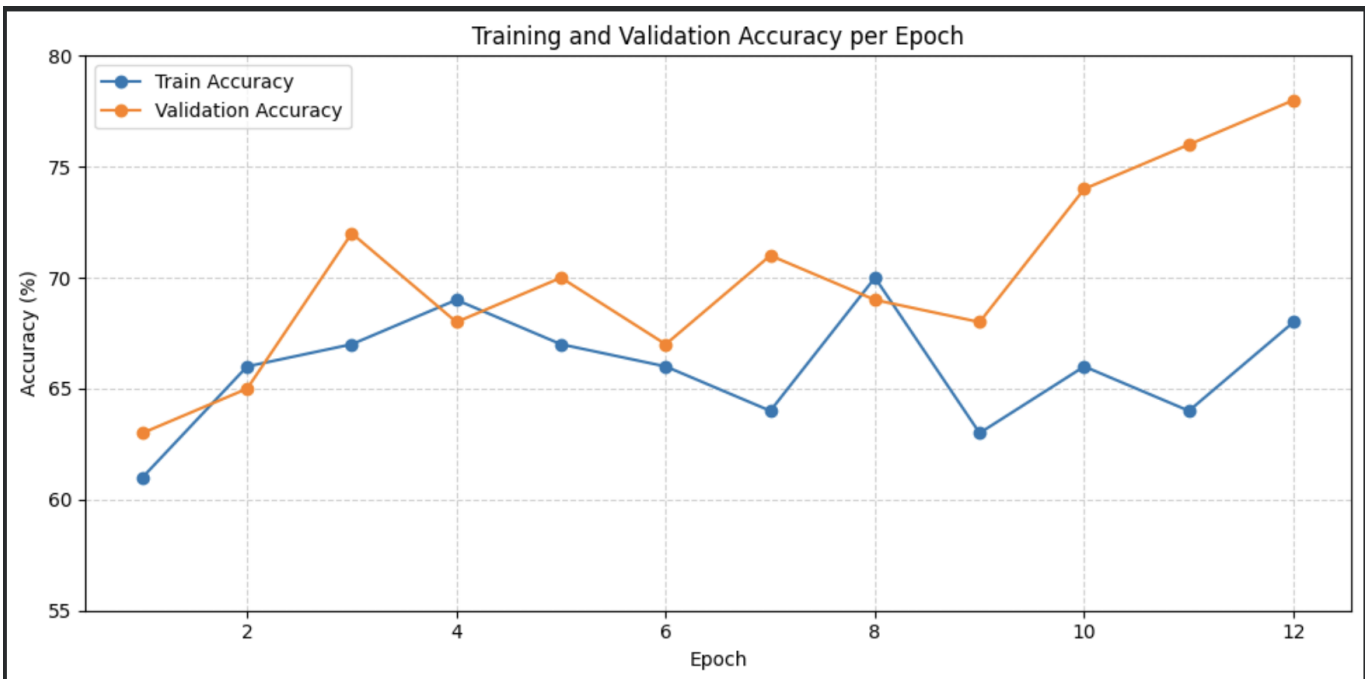
The validation accuracy remained consistently between 70–78%, demonstrating moderate but realistic generalization performance. Unlike earlier experiments that showed near-perfect (100%) validation scores

—likely due to overlapping samples or repeated validation images—this curve reflects true separation between training and validation data.

The gradual improvements confirms:

- Sampling with repetition provided better coverage of difficult negative pairs.
- ugmentation diversity increased robustness to unseen data.

The tuned margin threshold allowed better discrimination between positive and negative embeddings.

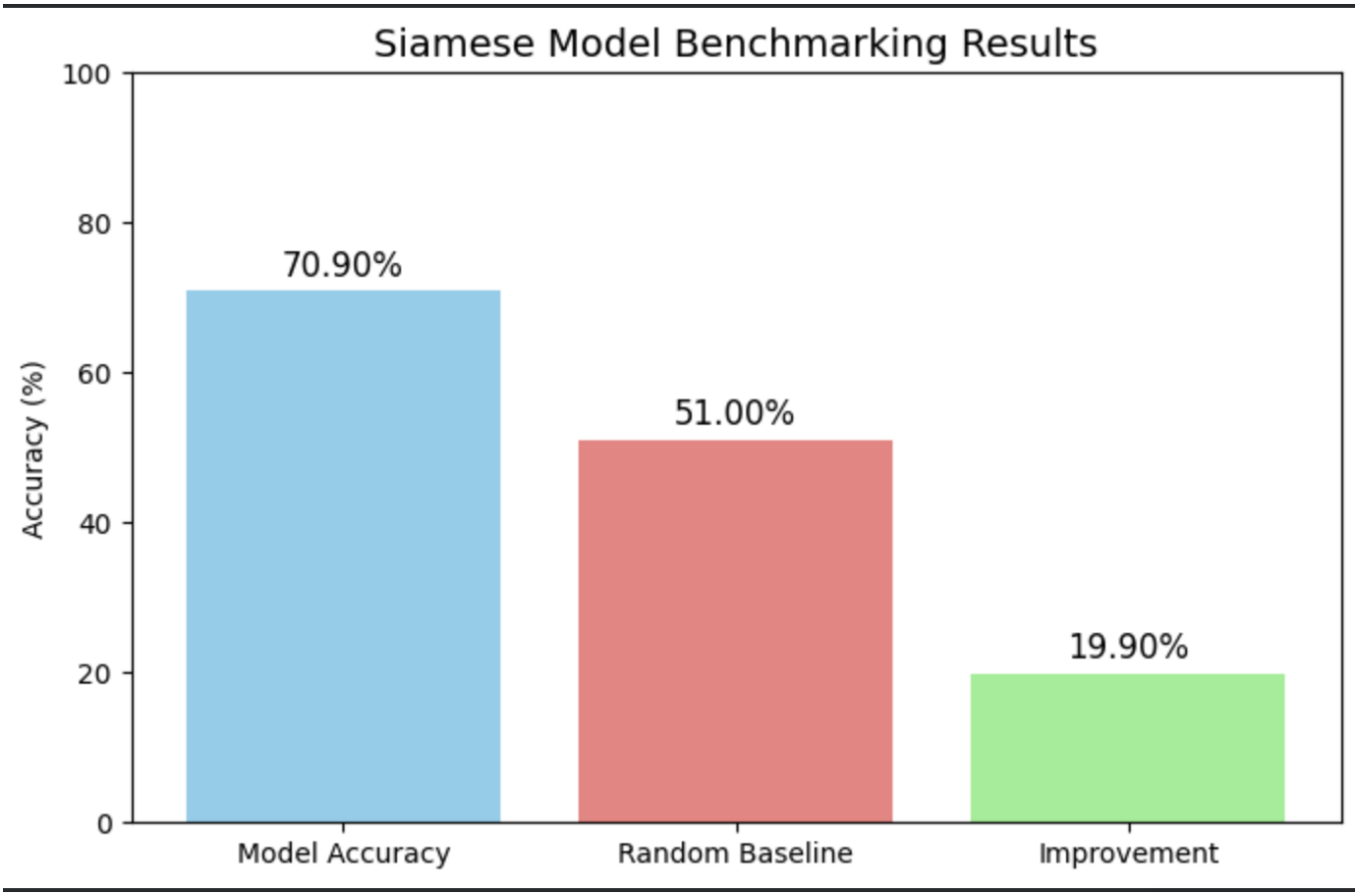


Model Benchmarking

To evaluate the performance of the Siamese Network on skin lesion similarity, I measured the model's accuracy on a precomputed validation set and compared it against a random baseline.

Metric: Binary similarity classification (predicted vs. actual pairs) Threshold: 0.5 (sigmoid output from network)

Metric Result Model Accuracy 70.90% Random Baseline 51.00% Improvement over Baseline 19.90%



The model correctly predicts whether two lesions belong to the same class 70.9% of the time, which is a 19.9% improvement over random chance.

This demonstrates that the network is learning meaningful representations of skin lesion similarity, far beyond what random guessing can achieve.

While the accuracy is not yet at the target 85%, it provides a solid foundation for further optimization, such as hyperparameter tuning, more balanced training pairs, or advanced augmentation strategies.

Run Instructions

Must have the images downloaded or the pki files

```
from google.colab import drive
drive.mount('/content/drive')

loader = DataLoader(width=105, height=105, channels=3,
                    data_path="/content/drive/MyDrive/subset_train",
                    csv_path="/content/drive/MyDrive/Groundtruth.csv",

output_path="/content/drive/MyDrive/siamese_isic_split.pkl")
loader.load()


!python siamese_train.py
```

Dependencies

Python 3.12+ TensorFlow 2.17+ NumPy Pandas Pillow Matplotlib scikit-learn

Further Improvements

Moving forward, I plan to experiment with advanced augmentations and possibly deeper network architectures to improve performance toward my goal of reaching higher accuracy. I also want to visualize the feature embeddings to better understand how the network separates benign and malignant lesions.